

Übung 10: Dynamische Programmierung I und II

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie können rekursive Brute-Force-Lösungen analysieren, redundante Teilprobleme erkennen und durch Memoization sowie den Bottom-Up-Ansatz effizientere Lösungen entwickeln.

1. Auf einer Zugstrecke gibt es N Bahnhöfe, die in der Reihenfolge $0, 1, 2, \dots, N - 1$ angefahren werden. Sie möchten von Bahnhof 0 nach Bahnhof $N - 1$ fahren. Für jedes Bahnhofspaar (i, j) mit $i < j$ sind die Kosten eines Tickets von Bahnhof i nach Bahnhof j gegeben.

Sie können mehrere Tickets kombinieren. Dabei muss das Ziel eines Tickets jeweils dem Startbahnhof des nächsten Tickets entsprechen. Gesucht ist eine Folge von Tickets, mit der Sie von Bahnhof 0 nach Bahnhof $N - 1$ gelangen und deren Gesamtkosten minimal sind.

Der folgende Code berechnet die minimalen Gesamtkosten für die Fahrt von Bahnhof 0 nach Bahnhof $N - 1$. Für das Beispiel mit $N = 4$ können Sie den Code ausführen und die berechneten minimalen Fahrtkosten bestimmen.

Zeigen Sie anhand des Rekursionsbaums, dass der Algorithmus dieselben Teilprobleme mehrfach berechnet. Zeichnen Sie dazu den vollständigen Rekursionsbaum der Methode `minCostRec` für das Beispiel $N = 4$. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `s` und `d` einzuzichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:



```

// A naive recursive solution to find min cost path from station 0 to station N-1
class shortest_path
{
    static int INF = Integer.MAX_VALUE, N = 4;
    // A recursive function to find the shortest path from source 's' to destination 'd'.
    static int minCostRec(int cost[][], int s, int d)
    {
        // If source is same as destination or destination is next to source
        if (s == d || s+1 == d)
            return cost[s][d];

        // Initialize min cost as direct ticket from source 's' to destination 'd'.
        int min = cost[s][d];

        // Try every intermediate vertex to find minimum
        for (int i = s+1; i<d; i++)
        {
            int c = minCostRec(cost, s, i) + minCostRec(cost, i, d);
            if (c < min)
                min = c;
        }
        return min;
    }

    // This function returns the smallest possible cost to
    // reach station N-1 from station 0. This function mainly uses minCostRec().
    static int minCost(int cost[][])
    {
        return minCostRec(cost, 0, N-1);
    }

    public static void main(String args[])
    {
        int cost[][] = { {0, 15, 80, 90},
                          {INF, 0, 40, 50},
                          {INF, INF, 0, 70},
                          {INF, INF, INF, 0}
                        };
        System.out.println("The Minimum cost to reach station "+ (N - 1) + " is " + minCost(cost));
    }
}

```

2. Gegeben sei eine positive ganze Zahl n . Bestimmen Sie die Anzahl der Binärzeichenfolgen der Länge n , die keine zwei aufeinanderfolgenden 1en enthalten.

Für $n = 5$ sind dies beispielsweise die folgenden Binärzeichenfolgen:

[00000, 00001, 00010, 00100, 00101, 01000, 01001, 01010, 10000, 10001, 10010, 10100, 10101].

Die Brute Force Lösung lautet wie folgt:

```
# zählt alle n-stelligen Binärzahlen ohne aufeinanderfolgende 1er
def countStrings(n, lastDigit=0):
    # Basisfall
    if n == 0:
        return 1

    #, wenn nur noch eine Ziffer übrig ist
    if n == 1:
        return 1 if (lastDigit == 1) else 2

    # Wenn die letzte Ziffer 0 ist, können wir an der aktuellen Position sowohl 0
    # als auch 1 haben
    if lastDigit == 0:
        return countStrings(n - 1, 0) + countStrings(n - 1, 1)
    # Wenn die letzte Ziffer 1 ist, können wir an der aktuellen Position nur 0 haben
    else:
        return countStrings(n - 1, 0)

if __name__ == '__main__':
    # Gesamtzahl der Ziffern
    n = 5

    print(f'The total number of {n}-digit binary numbers without any consecutive 1\'s'
          f' is {countStrings(n)}')
```

- (a) Zeichnen Sie den Rekursionsbaum der Funktion `countStrings` für $n = 5$. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente n und $lastDigit$ einzuzichnen.
- (b) Schreiben Sie unter jedes Blatt des Rekursionsbaums die Binärzeichenfolgen, die durch dieses Blatt repräsentiert werden.
- (c) Betrachten Sie den Rekursionsbaum aus der vorherigen Teilaufgabe. Welche unterschiedlichen Kombinationen der Parameter n und $lastDigit$ treten im Rekursionsbaum auf? Geben Sie diese als Menge von Paaren $(n, lastDigit)$ an. Welche dieser Kombinationen werden mehrfach berechnet?
- (d) Passen Sie den Code an, indem Sie Memoization verwenden.

3. Sie steigen eine Treppe mit n Stufen hinauf. Bei jedem Schritt können Sie entweder 1 oder 2 Stufen auf einmal steigen.

Auf wie viele verschiedene Arten können Sie die Treppe hinaufsteigen? Dabei zählt jede unterschiedliche Folge von Schritten als eine eigene Möglichkeit.

Beispiel: Für $n = 3$ gibt es 3 verschiedene Möglichkeiten:

$$[1, 1, 1], \quad [1, 2], \quad [2, 1].$$

Hinweis: Die Anzahl der Möglichkeiten erfüllt eine Rekursion, die eng mit der Fibonacci-Folge verwandt ist.

Die Brute Force Lösung lautet:

```
def climbingStairs(i, n):
    if i > n:
        return 0
    if i == n:
        return 1
    return climbingStairs(i + 1, n) + climbingStairs(i + 2, n)

if __name__ == '__main__':
    n = 4
    print(climbingStairs(0, n))
```

- (a) Zeichnen Sie den Rekursionsbaum für $n = 4$.
 (b) Machen Sie den Code effizienter, indem Sie Memoization nutzen.
 (c) Vervollständigen Sie die folgende Tabelle für $n = 8$. Dabei wird in `stair[i]` die Anzahl Möglichkeiten gespeichert eine Treppe mit i Stufen hochzugehen. Wir machen hier die Annahme, dass man eine Treppe mit null Stufen auf **eine** Art "hochgehen" kann.

i	0	1	2	3	4	5	6	7	8
<code>stair[i]</code>	1								

- (d) Geben Sie auf Grundlage Ihrer Tabelle einen iterativen Bottom-Up-Algorithmus in Pseudocode an.
4. Vergleichen Sie die folgenden drei Varianten des Treppenproblems:
- (a) rekursive Brute-Force-Lösung,
 - (b) rekursive Lösung mit Memoization,
 - (c) iterative Bottom-Up-Lösung.

Geben Sie für jede Variante die asymptotische Laufzeit und den asymptotischen zusätzlichen Speicherbedarf in Abhängigkeit von n an.

5. Sie sind für die Planung eines einmonatigen Sprints in einem Projekt verantwortlich. Für den Sprint steht eine begrenzte Ressourcenkapazität zur Verfügung. Ihnen liegt eine Liste von Aufgaben vor, die alle die gleiche Dauer (einen Monat) haben. Jede Aufgabe benötigt eine bestimmte Menge an Ressourcen und besitzt einen Prioritätswert.

Ziel ist es, eine Auswahl von Aufgaben zu treffen, deren Gesamtsumme der Prioritätswerte maximal ist, ohne die verfügbare Ressourcenkapazität zu überschreiten.

Für jede der 10 Aufgaben sind der Ressourcenbedarf (bspw. Anzahl Mitarbeiter) und der Prioritätswert gegeben. Entwickeln Sie einen rekursiven Brute-Force-Algorithmus in Pseudocode, der die maximal erreichbare Summe der Prioritätswerte bestimmt.

6. Das Problem der längsten gemeinsamen Teilfolge (*Longest Common Subsequence*, LCS) besteht darin, eine möglichst lange Folge zu finden, die sowohl eine Teilfolge der ersten als auch eine Teilfolge der zweiten gegebenen Sequenz ist. D.h., die längste Folge zu finden, die aus der ersten Originalsequenz durch Löschen einiger Elemente und aus der zweiten Originalsequenz durch Löschen einiger Elemente erhalten werden kann.

Beispiel: Gegeben die beiden Sequenzen

X: CBAD

Y: CAB

Die Länge einer LCS beträgt 2. Eine mögliche LCS ist CA.

Die Brute-Force Lösung lautet:

```
# Funktion zum Ermitteln der Länge der längsten gemeinsamen Teilfolge von
# Sequenzen 'X[0...m-1]' und 'Y[0...n-1]'
def LCSLength(X, Y, m, n):

    # kehrt zurück, wenn das Ende einer der beiden Sequenzen erreicht ist
    if m == 0 or n == 0:
        return 0

    #, wenn das letzte Zeichen von 'X' und 'Y' übereinstimmt
    if X[m - 1] == Y[n - 1]:
        return LCSLength(X, Y, m - 1, n - 1) + 1

    # andernfalls, wenn das letzte Zeichen von 'X' und 'Y' nicht übereinstimmt
    return max(LCSLength(X, Y, m, n - 1), LCSLength(X, Y, m - 1, n))

if __name__ == '__main__':

    X = 'CBAD'
    Y = 'CAB'

    print('The length of the LCS is', LCSLength(X, Y, len(X), len(Y)))
```

Zeigen Sie, dass dieser Code ineffizient ist, da die Funktion `LCSLength` häufig mit identischen Argumenten aufgerufen wird. Zeichnen Sie dazu den Rekursionsbaum der Methode `LCSLength` für das im Code aufgerufene Beispiel bis zur **maximalen Höhe von 3** (d.h. bis einschließlich zur dritten Rekursionsebene). Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `m` und `n` einzuzichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:

4, 3

7. Die Funktion `fib(n)` berechnet die n -te Fibonacci-Zahl mit einer rekursiven Brute-Force-Lösung. Die Funktion `fib_memo(n)` berechnet ebenfalls die n -te Fibonacci-Zahl, verwendet dabei jedoch Memoization, um bereits berechnete Ergebnisse wiederzuverwenden.

Wie viele Funktionsaufrufe führt `fib(5)` aus und wie viele Funktionsaufrufe führt `fib_memo(5)` aus? Eine Begründung ist nicht erforderlich. Es kann hilfreich sein, zuvor den Rekursionsbaum von `fib(5)` zu zeichnen.

```
public static int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n-1) + fib(n-2);
}
```

8. Das Partitionsproblem besteht darin, zu bestimmen, ob eine gegebene Liste positiver Ganzzahlen in zwei Teillisten aufgeteilt werden kann, sodass die Summe der Elemente in beiden Teillisten gleich ist. Beispielsweise kann die Liste `[4,2,2]` in die beiden Teillisten `[4]` und `[2,2]` aufgeteilt werden, da beide Summen den Wert 4 ergeben.

Die Brute-Force Lösung lautet:

```
def isSubsetSum(arr, n, total):
    if total == 0:
        return True
    if n == 0 and total != 0:
        return False

    # Das letzte Element ist grösser als die gesuchte Summe:
    # es kann nicht Teil einer gültigen Teilmenge sein, ignoriere es einfach
    if arr[n-1] > total:
        return isSubsetSum(arr, n-1, total)

    # Zwei Möglichkeiten, die mit OR verknüpft werden:
    # 1) arr[n-1] wird NICHT genommen -> suche 'sum' in den ersten n-1 Elementen
    # 2) arr[n-1] wird genommen -> suche 'sum - arr[n-1]' in den ersten n-1 Elementen
    return isSubsetSum(arr, n-1, total) or isSubsetSum(arr, n-1, total-arr[n-1])

def findPartition(arr, n):
    total = 0
    for i in range(0, n):
        total += arr[i]

    if total % 2 != 0:
        return False

    return isSubsetSum(arr, n, total // 2)

if __name__ == '__main__':
    arr = [4, 2, 2]
    n = len(arr)

    if findPartition(arr, n) == True:
        print("Can be divided into two subsets of equal sum")
    else:
        print("Can not be divided into two subsets of equal sum")
```

Zeichnen Sie den Rekursionsbaum der Funktion `isSubsetSum` für das im Code aufgerufene Beispiel. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `n` und `total` einzuzichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:



9. Für die folgenden Probleme soll entschieden werden, ob sich ein rekursiver Lösungsansatz sinnvoll mithilfe von dynamischer Programmierung verbessern lässt.

Geben Sie jeweils an, ob dynamische Programmierung geeignet ist, und begründen Sie Ihre Entscheidung kurz.

- (a) Berechnung der n -ten Fibonacci-Zahl.
- (b) Bestimmung des kürzesten Weges in einem gerichteten azyklischen Graphen.
- (c) Binäre Suche in einem sortierten Array.
- (d) Bestimmung der Länge einer längsten gemeinsamen Teilfolge zweier Sequenzen.

10. Die folgende Funktion berechnet die n -te Fibonacci-Zahl mithilfe von Memoization.

```
def fib(n, memo):
    if n <= 1:
        return n

    if memo[n] is not None:
        return memo[n]

    memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

- (a) Welche Zustände werden durch `memo` gespeichert?
- (b) In welcher Reihenfolge müssen diese Zustände berechnet werden, wenn die Rekursion durch einen Bottom-Up-Ansatz ersetzt werden soll?
- (c) Entwickeln Sie daraus einen iterativen Algorithmus zur Berechnung der n -ten Fibonacci-Zahl.